

Exercícios — Aula 05

Curso de Formação de Programação Aplicada · Módulo 1 · FPA Labs

Continue no projeto `api-filmes` da aula 04. Se preferir começar do zero, copie a solução da aula anterior.

Não esqueça de adicionar, logo depois de criar o `app`:

```
app.use(express.json());
```

Nível 1 — obrigatório

Exercício 1 — `POST /filmes`

Crie a rota que adiciona um filme.

Regras:

- Campos obrigatórios: `titulo`, `ano`, `genero`.
- Campos opcionais: `nota` (padrão `0`), `assistido` (padrão `false`).
- O **servidor** gera o `id` (o maior id existente + 1) e o campo `criadoEm`.
- Responde `201` com o filme criado.
- Se faltar campo obrigatório → `400` com a lista de erros.

Teste no `api.http` criando um filme e depois listando para ver se apareceu.

Exercício 2 — `DELETE /filmes/:id`

- Se o filme existir → remove e responde `204` sem corpo.
- Se não existir → `404` com mensagem clara.
- Se o id não for número → `400`.

Teste: apague, liste para confirmar que sumiu, e tente apagar de novo (deve dar `404`).

Exercício 3 — `PATCH /filmes/:id`

Atualização parcial: o cliente manda só o que quer mudar.

- Campos que podem ser alterados: `titulo`, `ano`, `genero`, `nota`, `assistido`.
- O `id` **nunca** muda, mesmo que venha no corpo.
- Adicione `atualizadoEm` com a data atual.
- Responde `200` com o filme atualizado.

⚠ Teste especificamente enviar `{ "assistido": false }`. Se o seu código usar `if (assistido)` em vez de `if (assistido !== undefined)`, esse teste vai falhar silenciosamente.

Nível 2 — esperado

Exercício 4 — Validação completa

Crie um arquivo `validacao.js` que exporta `validarNovoFilme(dados)` e `validarAtualizacao(dados)`. Cada uma devolve um **array de erros** (vazio = tudo certo).

Regras a validar:

Campo	Regra
<code>titulo</code>	obrigatório, texto, entre 1 e 200 caracteres
<code>ano</code>	obrigatório, número inteiro entre 1888 e o ano atual + 5
<code>genero</code>	obrigatório, um de: <code>drama</code> , <code>acao</code> , <code>comedia</code> , <code>suspense</code> , <code>terror</code> , <code>documentario</code>
<code>nota</code>	opcional, número entre 0 e 10
<code>assistido</code>	opcional, booleano

Responda **todos** os erros de uma vez:

```
{ "erros": ["ano deve ser um número entre 1888 e 2031", "genero inválido"] }
```

Exercício 5 — `PUT /filmes/:id`

Implemente a substituição total e **explique num comentário** a diferença prática para o `PATCH`.

Teste os dois com o mesmo corpo `{ "titulo": "Novo nome" }` num filme existente e compare o resultado. O `PUT` deve recusar (faltam campos obrigatórios); o `PATCH` deve aceitar.

Exercício 6 — Regra de negócio

Adicione uma regra que não é só validação de formato:

- **Não pode existir dois filmes com o mesmo título e o mesmo ano.** Se alguém tentar criar um duplicado, responda `409 Conflict` com uma mensagem explicando.

`409` é o status para "o pedido está correto, mas conflita com o estado atual dos dados". Vale pesquisar outros casos em que ele é usado.

Desafio — opcional

Exercício 7 — Sub-recurso: avaliações

Um filme pode ter várias avaliações. Implemente:

Rota	O que faz
GET /filmes/:id/avaliacoes	lista as avaliações daquele filme
POST /filmes/:id/avaliacoes	adiciona uma avaliação ({ autor, nota, comentario })
DELETE /filmes/:filmeId/avaliacoes/:id	remove uma avaliação

Requisitos:

- Guarde as avaliações num array separado, com o campo `filmeId` ligando ao filme.
- `nota` deve ser um número inteiro de 1 a 5.
- Se o filme não existir, todas as rotas respondem `404`.
- Ao adicionar ou remover uma avaliação, **recalcule** a nota média do filme.

Repare no desenho da URL: `/filmes/3/avaliacoes` lê-se naturalmente como "as avaliações do filme 3". Isso é REST bem feito.

Exercício 8 — Middleware de log

Escreva um middleware caseiro que registre, para cada requisição:

```
[14:32:07] POST /filmes      201 · 4ms
[14:32:11] GET  /filmes/999  404 · 1ms
```

Dicas: registre `Date.now()` no começo; use `res.on("finish", ...)` para saber quando a resposta terminou; `res.statusCode` tem o status final. Não esqueça de chamar `next()`.

Checklist de entrega

- ☐ `app.use(express.json())` está no arquivo
- ☐ `POST` responde `201`, `DELETE` responde `204`
- ☐ Validação recusa dados ruins com `400` e lista todos os erros
- ☐ Id inexistente responde `404` em todas as rotas que recebem `:id`
- ☐ Testei enviar `false` num campo booleano e funcionou
- ☐ Meu `api.http` cobre o ciclo completo e os casos de erro
- ☐ O `id` é sempre gerado pelo servidor, nunca aceito do cliente

